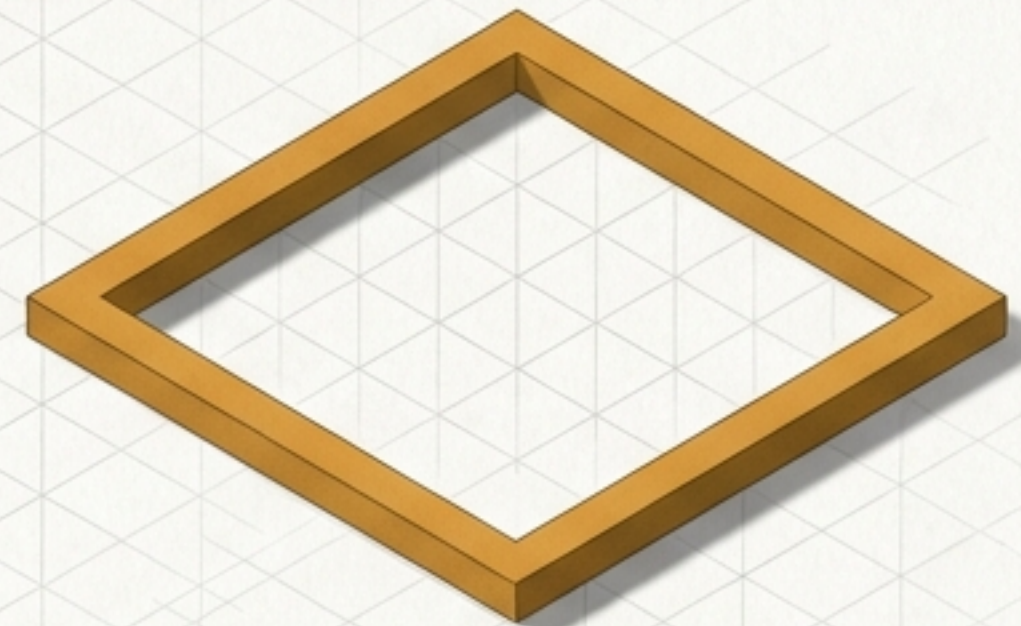
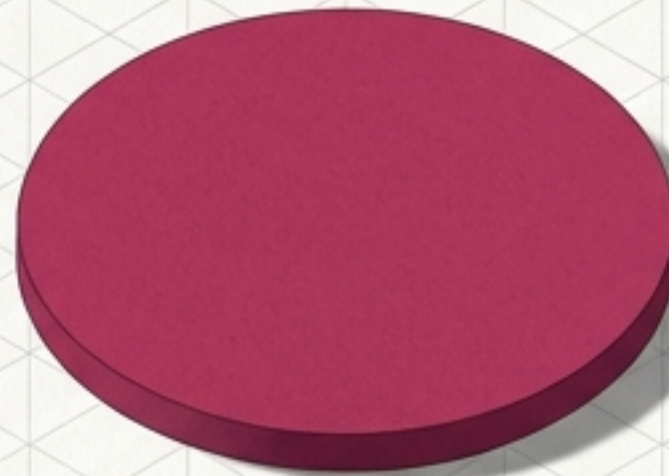
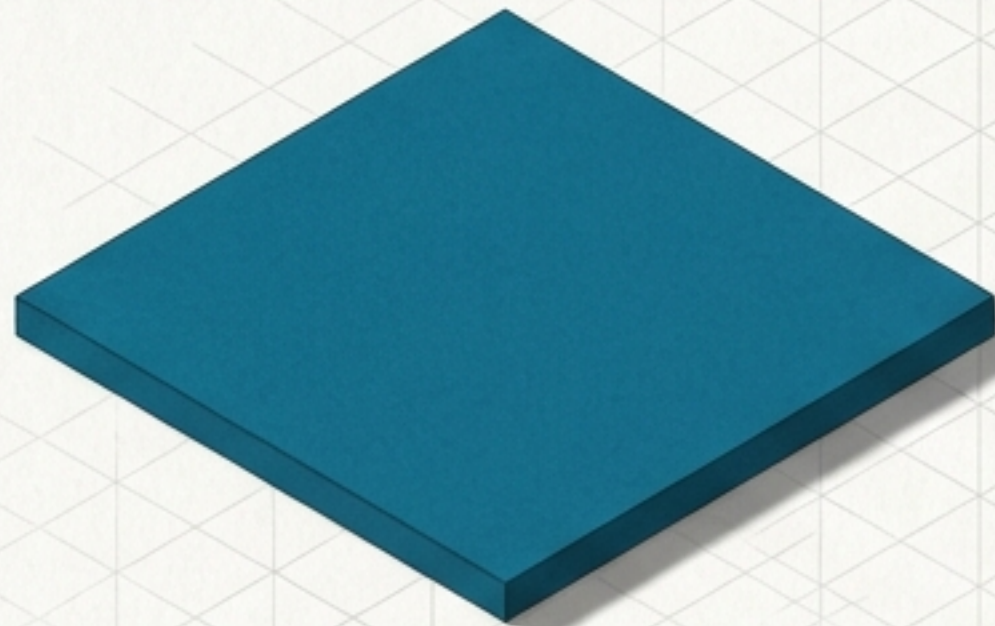


Building Systems Over Writing Scripts



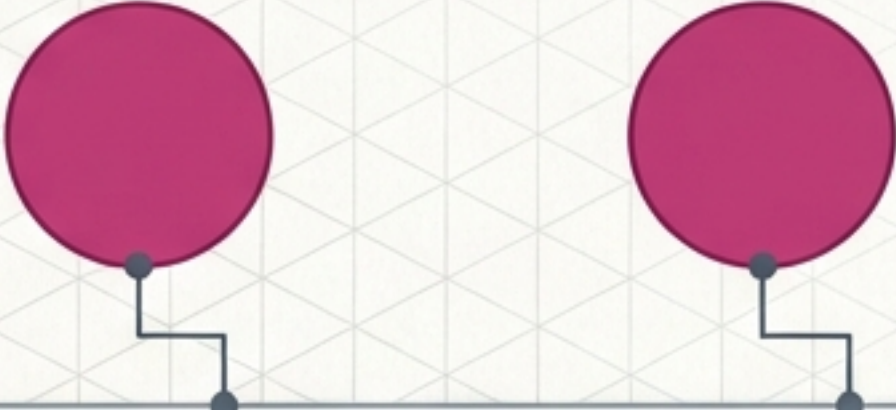
A visual guide to the natural evolution of state, behavior, and structural logic in modern TypeScript applications.

The Procedural Baseline Separates Form from Function

```
type User = {  
  name: string;  
  age: number;  
};
```



The **Form**: Raw data structures passed globally.



```
function makeOlder(user: User): User {  
  return { ...user, age: user.age + 1 };  
}  
  
function printUser(user: User) {  
  console.log(`${user.name} is ${user.age}`);  
}
```

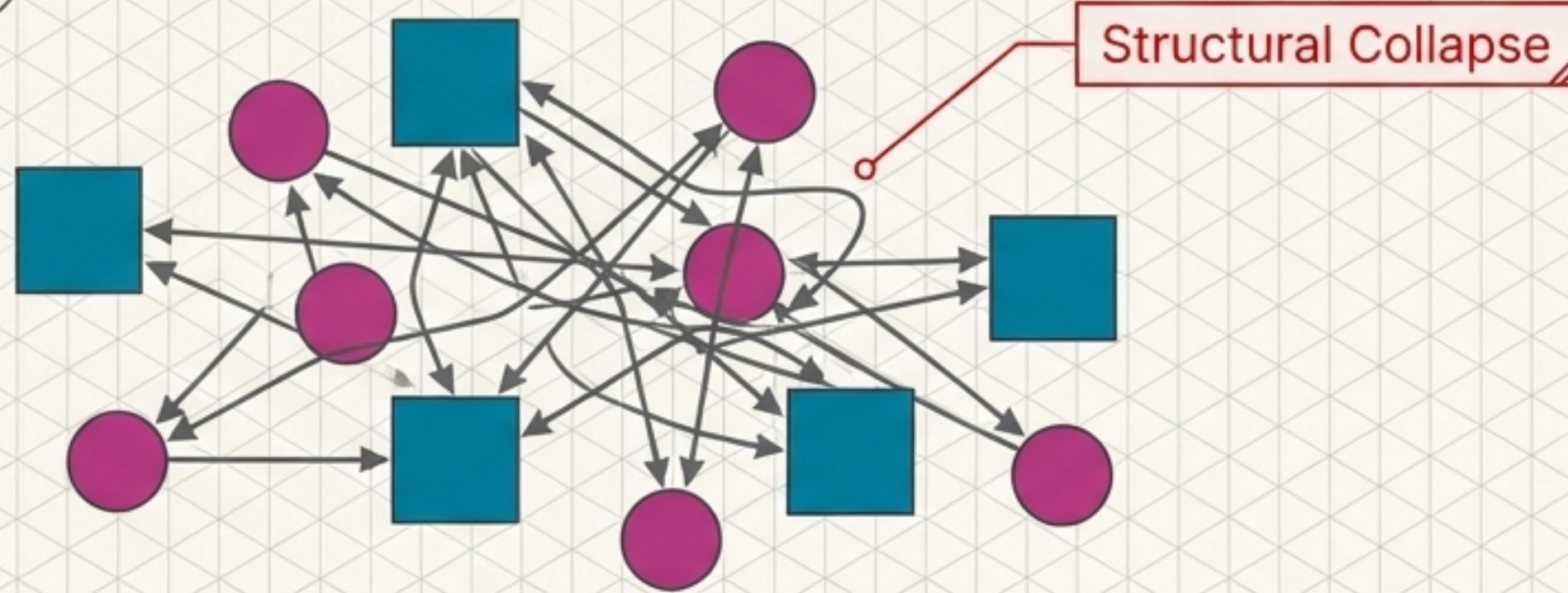
Two magenta circles representing standalone logic (Function) disconnected from state. Each circle is connected by a line to a callout box below it.

The **Function**: Standalone logic disconnected from state.

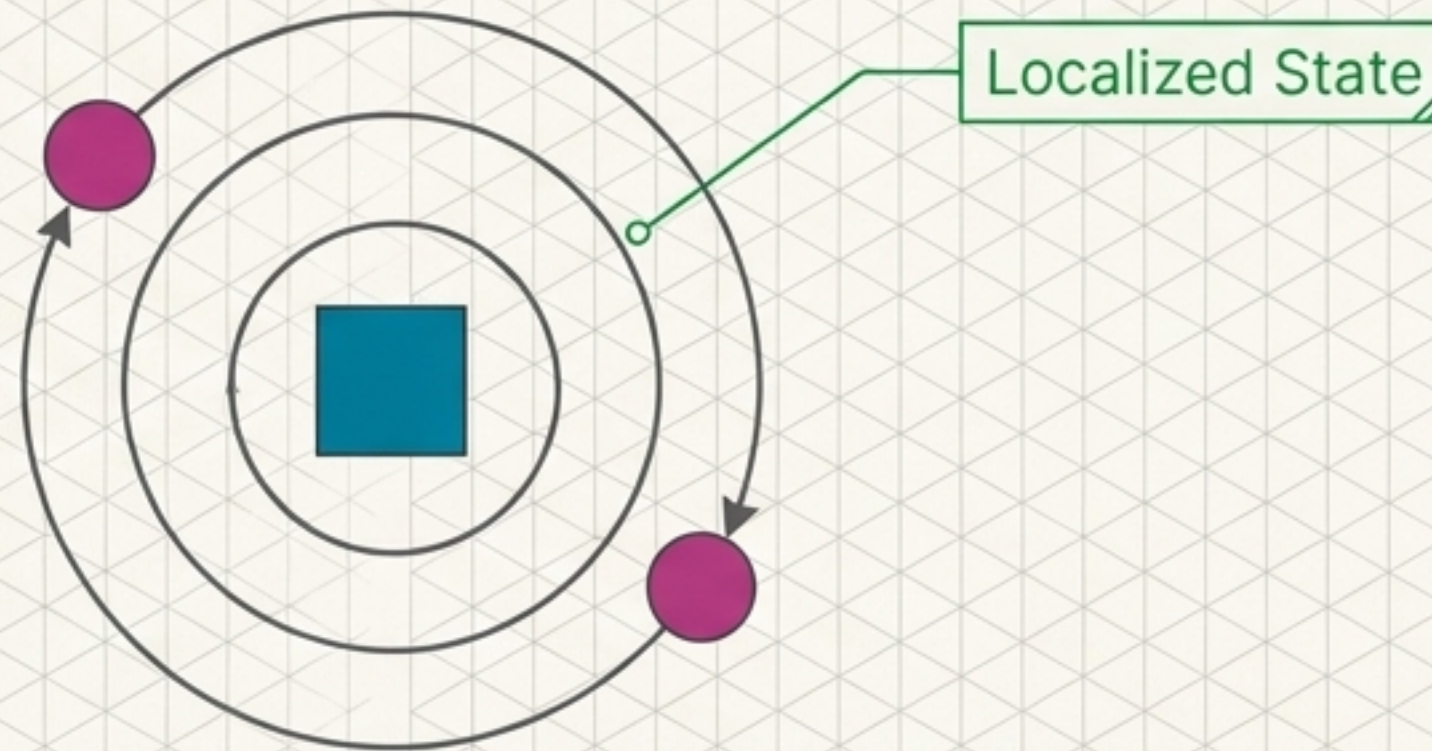
A simple data-plus-functions approach works for small scripts, but form and function remain functionally unaware of each other.

Logic Scatters as the System Grows

Spaghetti Logic



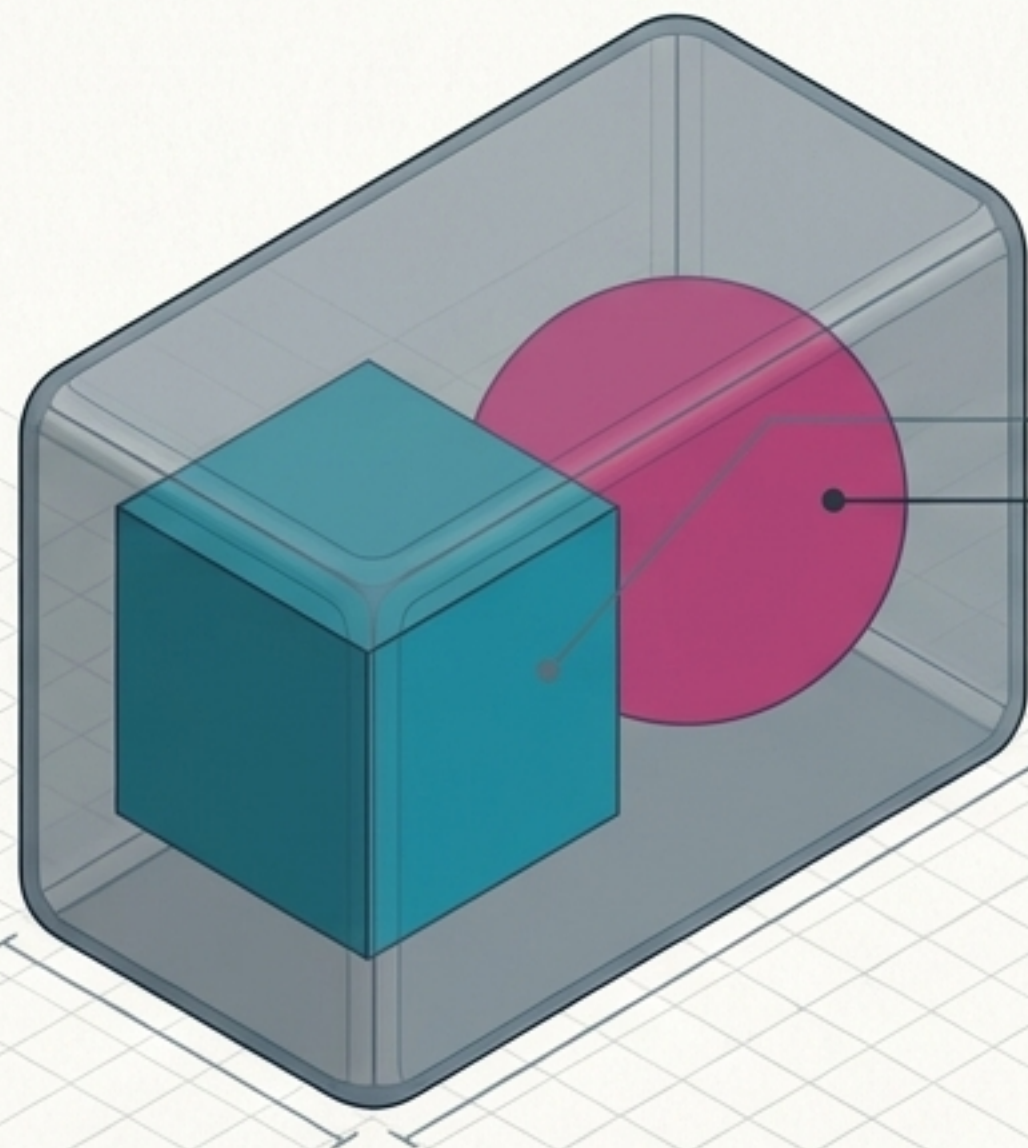
Object Gravity



Scaling Friction Points

1. Where is the behavior located? ●
2. Who actually owns the logic? ●
3. How do we prevent side-effects? ●

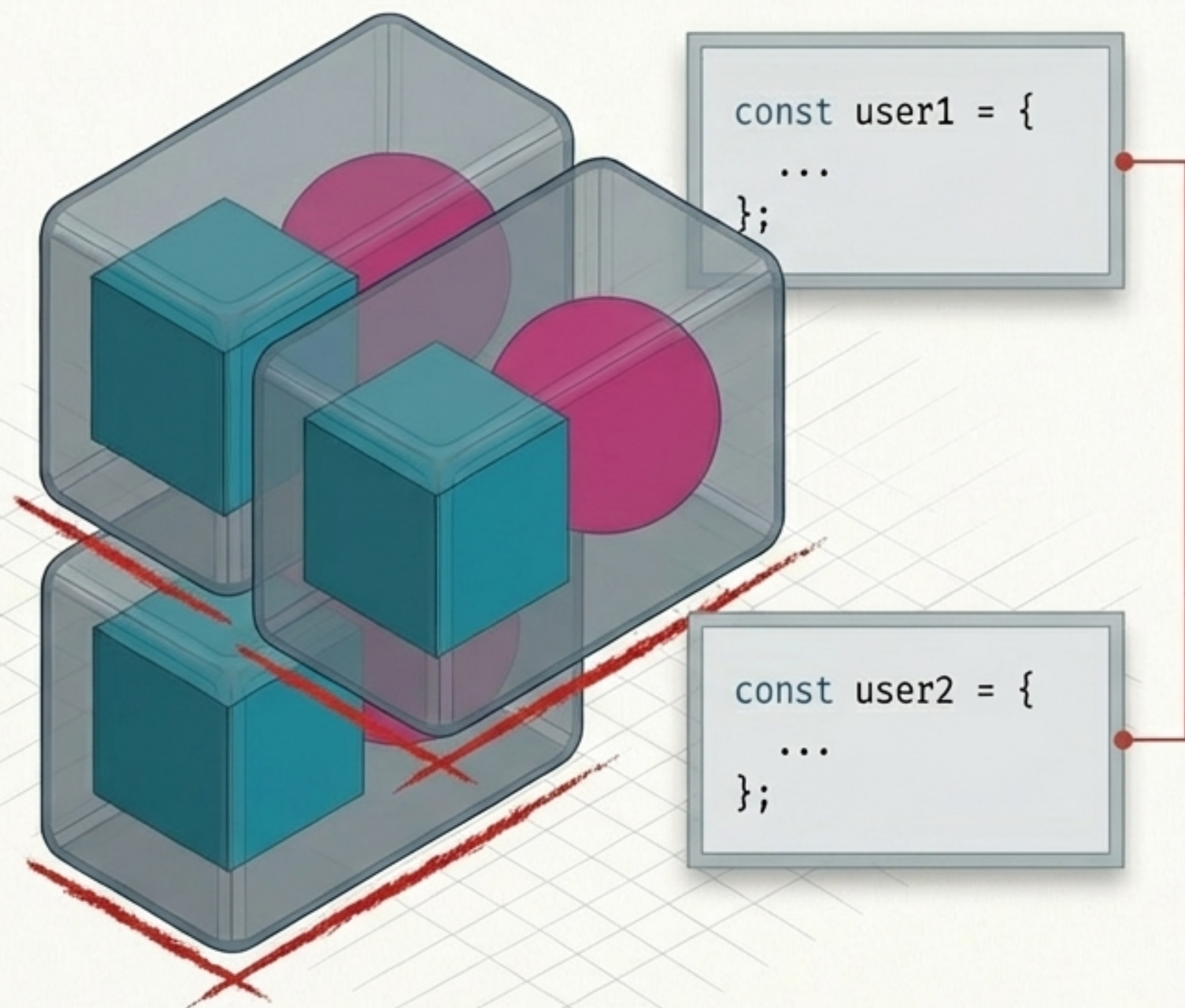
Merging Data and Behavior Establishes Ownership



```
const user = {  
  name: "Otavio",  
  age: 30,  
  makeOlder() { this.age++; },  
  print() { console.log(`$this.name is ${.age}`); },  
};
```

Synthesis: Behavior now lives natively with the data. There is no longer a need to pass the object into external, disconnected functions.

Structural Repetition Kills Scalability



Scalability Warning Matrix



Repetition: Identical logic is unnecessarily duplicated in memory for every single instance.



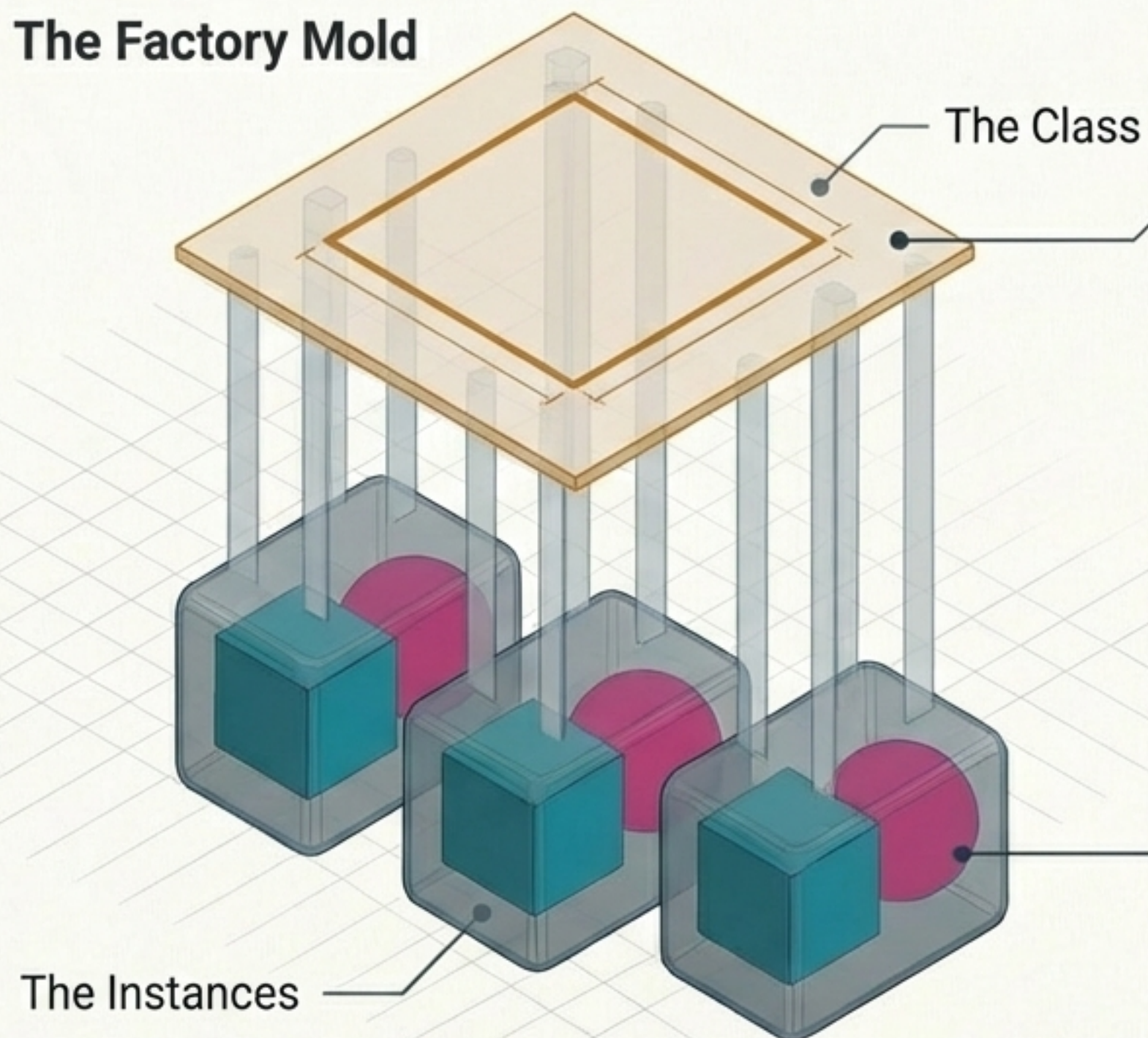
Fragility: No shared underlying structural contract exists to guarantee consistency.



Friction: Manual duplication becomes geometrically harder to scale across a growing codebase.

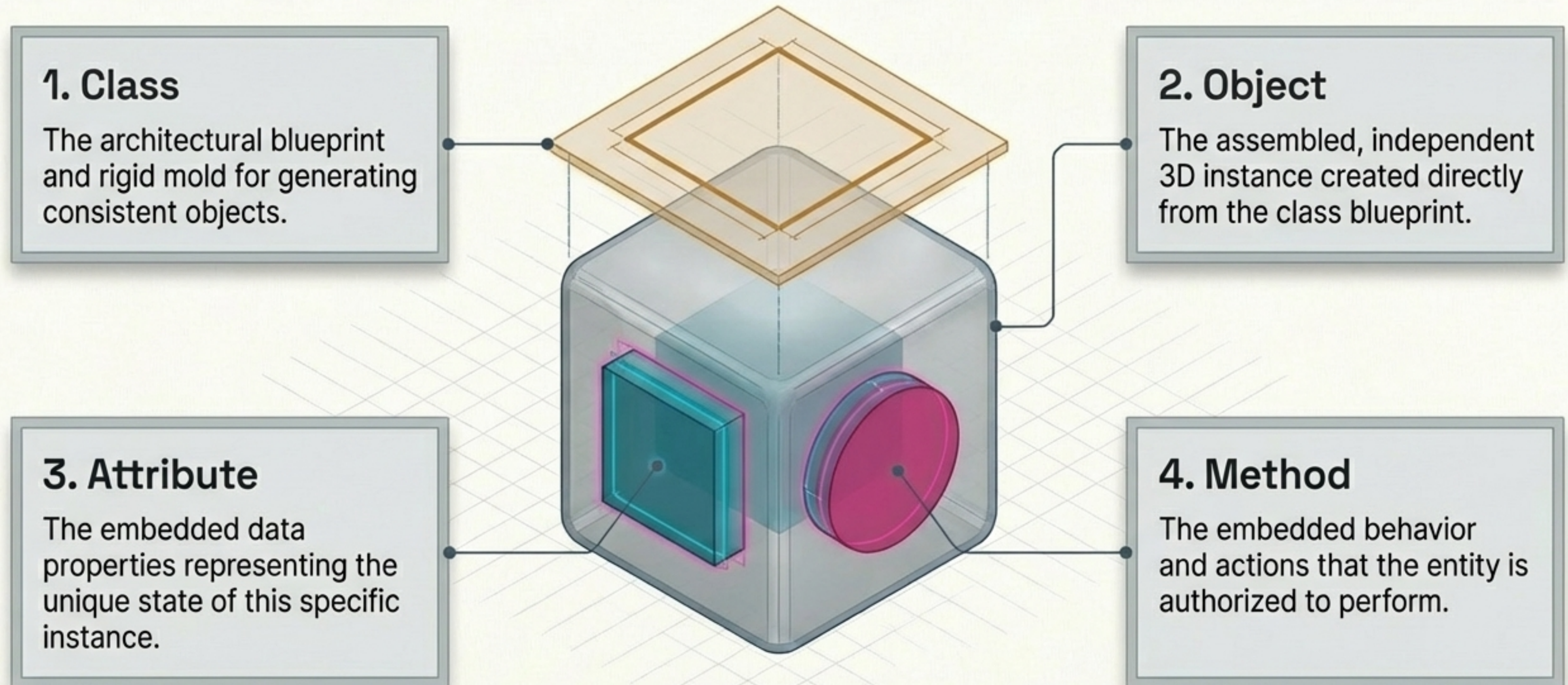
Standardizing Creation with Architectural Blueprints

The Factory Mold

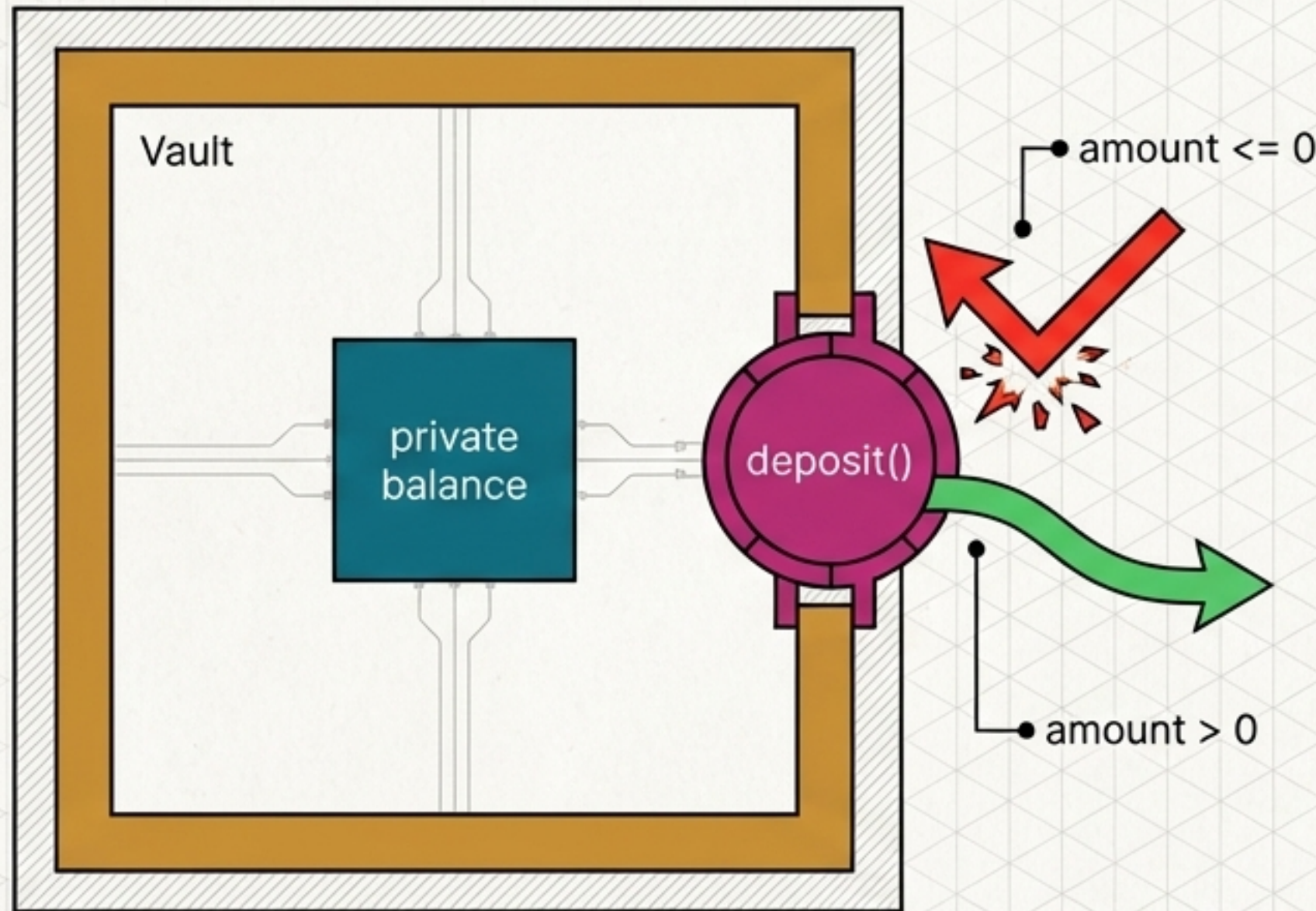


```
class User {  
  constructor(public name: string,  
    public age: number) {}  
  makeOlder() { this.age++; }  
  print() { console.log(`${this.name}  
    is ${this.age}`);  
}  
  
// Creating perfectly stamped objects  
const u1 = new User("Otavio", 30);  
const u2 = new User("Ana", 25);
```


Anatomy of a Structured System



Protecting Invariants via Encapsulation



```
class BankAccount {  
    private balance = 0;  
    deposit(amount: number) {  
        if (amount <= 0) throw new Error("Invalid");  
        this.balance += amount;  
    }  
}
```

Encapsulation protects core business rules (invariants). It dictates exactly how data is modified, keeping invalid external states strictly outside the membrane.

The Architectural Tradeoff

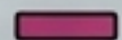


Functional Approach

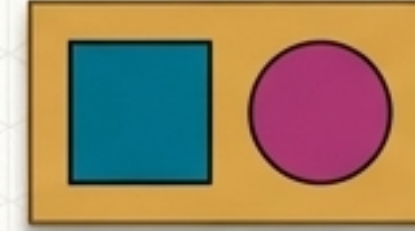
```
deposit(balance, amount) {  
  return balance + amount;  
}
```



Strengths: Extremely simple to test and highly predictable.



Weaknesses: Logic easily scatters across the system without clear ownership.



Object-Oriented Approach

```
account.deposit(100);
```



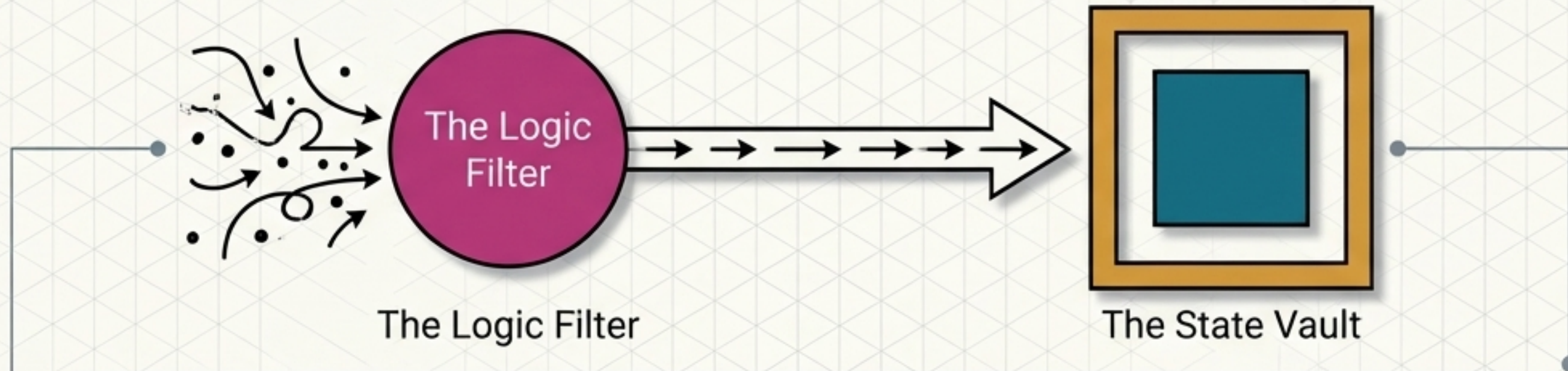
Strengths: Tightly groups related behavior directly with its specific data.



Weaknesses: Can become deeply rigid, state-heavy, and bloated if overused.

Architectural Verdict: Neither paradigm is perfect in isolation.
Scaling requires synthesizing both approaches.

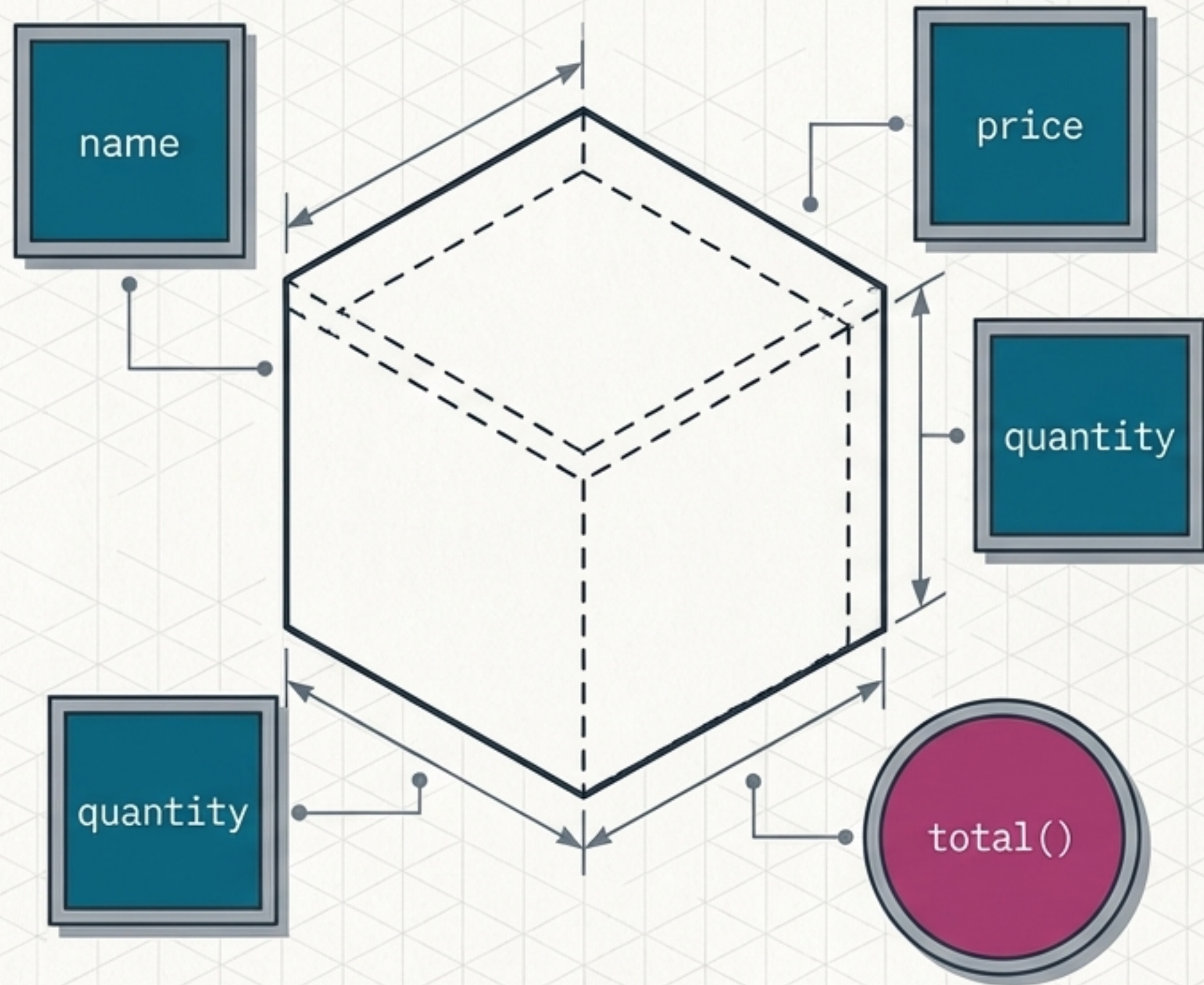
The Synthesis: Pure Logic Meets Stateful Vaults



```
class BankAccount {  
  private balance = 0;  
  deposit(amount: number) {  
    // Hybrid execution: Pure function filters, Class holds state  
    this.balance = deposit(this.balance, amount);  
  }  
}
```

Core Architectural Rule: Use pure functional pipelines for core business logic validation. Use class-based vaults to securely organize and mutate the resulting state.

Blueprint Application: The Cart Item



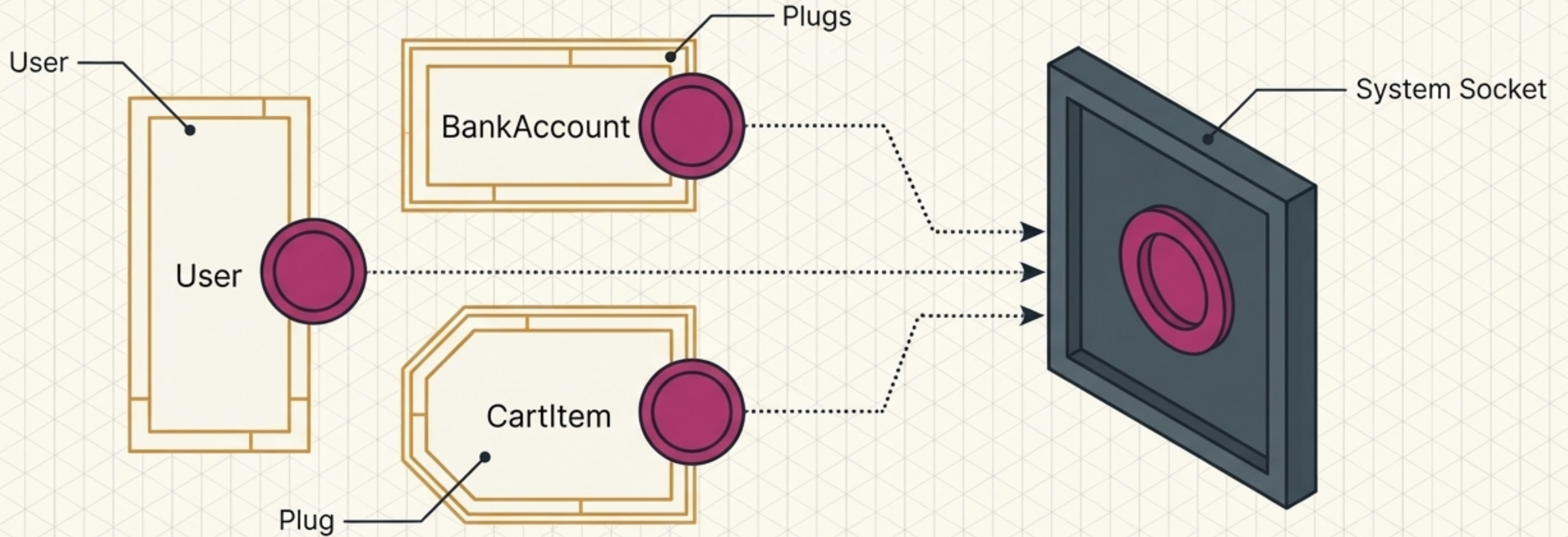
Model a CartItem System

Will you engineer a functional pipeline or encapsulate within a class-based vault?

Which structural approach guarantees absolute clarity for the next developer?

Which approach resists structural collapse when discount rules change?

The Next Horizon: Depending on Behavior



We have mastered wrapping data and logic into rigid blueprints.

But how do we use radically different objects interchangeably without endless conditionals?

What if our systems depended purely on behavior contracts, rather than concrete types?